Karpathy LLM-Wiki

> **基于**：Andrej Karpathy的LLM-Wiki方法论
>
> **编写日期**：2026年4月8日
>
> **版本**：v1.0
>
> **文档性质**：技术选型

📑 目录

- 第一部分：核心理念
- 第二部分：LLM-Wiki Skill实现
- 第三部分：LLM-Wiki开源实现
- 第四部分：方案对比与选型建议

第一部分：核心理念

1.1 什么是LLM-Wiki?

定义

LLM Wiki 是一种使用大语言模型（LLM）构建和维护**个人知识库**的创新模式，由Andrej Karpathy于2026年4月提出。

核心思想

> 让AI持续编译和维护你的知识库，而不是每次查询时重新检索

1.2 传统RAG的局限性

传统RAG工作流程

```
┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ 上传文档 │──▶│ 向量索引 │──▶│ 检索片段 │──▶│ 拼凑答案 │
└──────────┘    └──────────┘    └──────────┘    └──────────┘
```

核心问题

问题	说明
❌ **无积累**	每次查询从头开始
❌ **无综合**	只能拼凑片段，无法形成整体认知
❌ **无记忆**	好的答案消失在聊天历史中
❌ **无进化**	知识库不会随时间改进
❌ **无连接**	文档之间的关系需要每次重新发现

实际体验

```
第1次提问："Transformer架构的原理？"
    → LLM检索论文片段 → 回答

第2次提问："Transformer vs RNN的优缺点？"
    → 又一次检索 → 又一次拼凑 → 回答

第3次提问："BERT是如何使用Transformer的？"
    → 再一次从头开始...

❌ 之前的推理过程完全丢失
❌ 知识之间的连接需要重新建立
```

1.3 LLM-Wiki的创新之处

工作流程

```
┌─────────┐     ┌─────────┐     ┌─────────┐     ┌─────────┐
│ 上传文档 │──→│ AI阅读  │──→│ 编译Wiki │──→│ 持久维护 │
└─────────┘     └─────────┘     └─────────┘     └─────────┘
      ↑                                                  │
      └──────────────── 更新与扩展 ←──────────────────┘
```

核心优势

优势	说明
✅ **知识编译**	文档被编译成结构化Wiki页面
✅ **持续积累**	每个新文档让Wiki更丰富
✅ **自动维护**	AI自动更新、链接、检查矛盾
✅ **复利增长**	知识库价值随时间指数增长
✅ **交叉引用**	自动建立和维护文档间连接

1.4 三层架构设计

架构图

```
┌──────────────────────────────────────────────────────────┐
│                    第一层：Raw Sources                    │
│          (PDF、文章、笔记、代码、图片、视频转录)          │
│                 不可变的真相之源（Truth Layer）            │
└──────────────────────────────────────────────────────────┘
                          │
                          ↓
┌──────────────────────────────────────────────────────────┐
│                    第二层：The Wiki                       │
│        (AI生成的Markdown页面、摘要、实体页、概念页)       │
│                 由LLM完全控制（Knowledge Layer）           │
└──────────────────────────────────────────────────────────┘
                          │
                          ↓
┌──────────────────────────────────────────────────────────┐
│                  第三层：The Schema/Tools                 │
│           (规则文档、索引、日志、搜索工具)                 │
│               指导LLM行为（Governance Layer）             │
└──────────────────────────────────────────────────────────┘
```

层级详解

第一层：原始资料层（Raw Sources）

特性	说明
定位	不可变的真相之源
内容	PDF、网页、笔记、代码、图片等
权限	只读，LLM从不修改
作用	提供原始事实依据

第二层：Wiki层（The Wiki）

特性	说明
定位	AI维护的知识网络
内容	摘要、实体页、概念页、比较页、综合分析
权限	LLM完全控制，自动更新
作用	提供结构化、互联的知识表示

第三层：Schema/工具层（The Schema/Tools）

特性	说明
定位	治理与工具支持
内容	规则文档、索引、日志、搜索引擎、CLI工具
权限	人类定义规则，LLM遵循
作用	确保一致性和可扩展性

1.5 四大核心操作

操作流程图

```
┌──────────┐
│  Ingest  │  导入新资料
│  (导入)  │──────────┐
└──────────┘          │
                      ↓

┌──────────┐      ┌──────────┐
│  Query   │      │  Wiki    │
│  (查询)  │◀─────│  知识库  │
└──────────┘      └──────────┘

     │                  ↑
     │ (File Back)      │
     └──────────────────┘

                      ↑
┌──────────┐          │
│   Lint   │  健康检查  │
│  (自检)  │──────────┘
└──────────┘
```

---

1. Ingest（导入）

流程：

```
Step 1: 上传文档到原始资料集合
    ↓
Step 2: LLM读取并与你讨论关键要点
    ↓
Step 3: 生成文档摘要页面
    ↓
Step 4: 更新索引文件
    ↓
Step 5: 创建或更新相关实体页、概念页
    ↓
Step 6: 标记与现有知识的矛盾
    ↓
Step 7: 在日志中记录本次导入
```

影响范围：单个文档可能触及10-15个Wiki页面

---

2. Query（查询）

流程：

```
Step 1: 用户提问
    ↓
Step 2: LLM搜索相关Wiki页面（通过索引或搜索工具）
    ↓
Step 3: 读取相关页面
    ↓
Step 4: 综合答案并附上引用
    ↓
Step 5: （可选）将好答案归档回Wiki
```

输出形式：

- Markdown文档
- 比较表格
- 幻灯片（Marp）
- 图表（matplotlib）
- Canvas画布

---

3. Lint（自检）

检查项目：

检查内容	说明	自动化
🔍 矛盾检查	页面之间的矛盾声明	✅
📅 过时声明	被新资料取代的信息	✅
🚫 孤儿页面	没有入站链接的页面	✅
🏷️ 缺失页面	被提及但未创建的概念	✅
🔗 缺失交叉引用	应该存在的链接	✅
🌐 数据空白	可通过网络搜索填补的空白	⚠️

---

4. File Back（回填）

理念：

> 好的答案不应该消失在聊天历史中

机制：

```
优质查询答案 → 归档为新Wiki页面 → 成为知识库一部分 → 未来查询可引用
```

结果：

```
你的探索过程本身也在为知识库复利增长
```

第二部分：LLM-Wiki Skill实现

2.1 项目概述

项目定位

LLM-Wiki Skill 是一个基于 Karpathy 的 llm-wiki 方法论**轻量级的Agent技能/插件**，为AI编程助手（如Claude Code、Codex、OpenClaw）提供知识库管理能力。

基本信息

项目	信息
类型	Agent技能/CLI工具
目标用户	个人开发者、研究人员
部署方式	本地安装
界面	无UI，通过Obsidian查看
数据存储	本地文件系统（Markdown）
参考地址	https://github.com/sdyckjq-lab/llm-wiki-skill

2.2 技术架构

系统架构图

```
┌──────────────────────────────────────────────────────────┐
│              AI编程助手 (Claude Code/Codex)               │
│                   ↓ 调用技能                              │
│          ┌──────────────────────────────┐                 │
│          │    LLM-Wiki-Skill (Bash脚本)    │                 │
│          └──────────────┬───────────────┘                 │
│                         ↓                                  │
│          ┌──────────────────────────────┐                 │
│          │   内容处理工具集                │                 │
│          │  - baoyu-url-to-markdown       │                 │
│          │  - wechat-article-to-markdown  │                 │
│          │  - yt-dlp                      │                 │
│          │  - ...                         │                 │
│          └──────────────┬───────────────┘                 │
│                         ↓                                  │
│          ┌──────────────────────────────┐                 │
│          │   本地文件系统                  │                 │
│          │   raw-sources/  ← 原始资料      │                 │
│          │   wiki/        ← AI生成Wiki    │                 │
│          │   AGENTS.md    ← Schema配置    │                 │
│          └──────────────┬───────────────┘                 │
│                         ↓                                  │
│          ┌──────────────────────────────┐                 │
│          │   Obsidian （查看和导航）       │                 │
│          └──────────────────────────────┘                 │
└──────────────────────────────────────────────────────────┘
```

目录结构

```
llm-wiki-project/
├── raw-sources/              # 第一层：原始资料
│   ├── web/                 # 网页文章
│   ├── x-twitter/           # X/Twitter内容
│   ├── wechat/              # 微信文章
│   ├── youtube/             # YouTube转录
│   ├── papers/              # 学术论文
│   ├── books/               # 书籍
│   ├── podcasts/            # 播客
│   ├── documents/           # 文档
│   └── assets/              # 图片等附件
│
├── wiki/                     # 第二层：AI生成Wiki
│   ├── sources/             # 文档摘要
│   ├── entities/            # 实体页面
│   │   ├── people/
│   │   ├── concepts/
│   │   ├── tools/
│   │   └── organizations/
│   ├── topics/              # 主题综述
│   ├── comparisons/         # 对比分析
│   ├── synthesis/           # 综合分析
│   ├── index.md            # 索引文件
│   └── log.md              # 日志文件
│
├── AGENTS.md                 # Schema配置（Claude Code用）
├── .cursorrules             # Schema配置（Cursor用）
├── .clawconfig              # Schema配置（OpenClaw用）
│
└── .obsidian/               # Obsidian配置
    └── config/
```

2.3 支持的内容来源

九大内容来源

序号	内容源	处理工具	输出格式
1	**Web网页**	baoyu-url-to-markdown	Markdown + 图片
2	**X/Twitter**	baoyu-url-to-markdown	Markdown
3	**微信公众号**	wechat-article-to-markdown	Markdown + 图片
4	**YouTube视频**	yt-dlp + Whisper	转录文本
5	**学术论文**	手动上传PDF	PDF
6	**书籍**	手动上传	PDF/EPUB
7	**播客**	yt-dlp（音频）+ Whisper	转录文本
8	**文档**	手动上传	PDF/Word/TXT
9	**个人笔记**	直接创建	Markdown

2.4 模型付费

Claude Code（推荐）

套餐	费用	特性
Claude Pro		消息限制较少，优先访问新模型
Claude API		更灵活，适合重度使用

估算成本：

OpenAI Codex

套餐	费用	特性
ChatGPT Plus		包含Code Interpreter
OpenAI API		GPT-4: $0.03/1K tokens (input)

估算成本：

本地模型（免费）

2.5 硬件要求

基础配置（云端模型）

组件	最低要求	推荐配置
CPU	双核 2GHz	四核 3GHz+
内存	8GB	16GB+
存储	10GB可用空间	50GB+ SSD
网络	稳定网络连接	10Mbps+

本地模型配置

2.6 优缺点分析

优势

优势	说明
✅ **简单轻量**	Bash脚本，易于理解和修改
✅ **零依赖**	无需数据库、无需Web服务器
✅ **本地优先**	数据完全在本地，隐私安全
✅ **Git友好**	纯文本文件，版本控制完美支持
✅ **工具集成**	与Obsidian、VSCode等无缝配合
✅ **内容丰富**	支持9种内容来源
✅ **成本低**	仅需API费用，无服务器成本
✅ **灵活性强**	易于定制和扩展

劣势

劣势	说明
❌ **无Web界面**	依赖Obsidian查看
❌ **扩展性受限**	索引文件在数千页面时会溢出
❌ **单用户**	不支持多用户协作
❌ **手动搜索**	无内置搜索引擎
❌ **需要技术背景**	需要命令行基础
❌ **移动端受限**	Obsidian移动版功能有限
❌ **缺少云同步**	需手动配置（如iCloud、Dropbox）

第三部分：LLM-Wiki开源实现

3.1 项目概述

项目定位

LLM-Wiki开源实现 是Karpathy方法论的**完整Web应用实现**，提供企业级的知识管理平台。

基本信息

项目	信息
类型	全栈Web应用
目标用户	
部署方式	自托管
界面	现代化Web UI
数据存储	PostgreSQL + S3
参考地址	https://github.com/lucasastorian/llmwiki
官方服务	https://llmwiki.app/

3.2 设计架构

技术架构

```
┌──────────────────────────────────────────────────────────────┐
│                        Claude.ai                             │
│                          ↓ MCP协议                           │
└──────────────────────────┬───────────────────────────────────┘
                           ↓
┌──────────────────────────────────────────────────────────────┐
│                     MCP Server (Python)                      │
│           提供5个工具：guide/search/read/write/delete        │
└──────────────────┬───────────────────────────────────────────┘
                   ↓
┌──────────────────────────────────────────────────────────────┐
│              Next.js 16 Web Frontend (React 19)              │
│  - Dashboard  - Files  - Wiki  - Settings                    │
└──────────────────┬───────────────────────────────────────────┘
                   ↓
┌──────────────────────────────────────────────────────────────┐
│                FastAPI Backend (Python 3.11+)                │
│  - 认证授权  - 文件上传  - 文档处理  - OCR                   │
└──────────┬──────────────────────────────┬────────────────────┘
           ↓                              ↓
┌────────────────────┐    ┌────────────────────────┐
│   Supabase         │    │   S3 Storage           │
│  (PostgreSQL)      │    │  (MinIO/R2/AWS)        │
│  - 用户数据        │    │  - 原始文件            │
│  - 文档元数据      │    │  - Wiki页面            │
│  - PGroonga搜索    │    │  - 图片附件            │
└────────────────────┘    └────────────────────────┘
```

四层架构

```
┌──────────────────────────────────────────────────────────┐
│  Presentation Layer (展示层)                              │
│  Next.js Frontend + MCP Tools for Claude                  │
└──────────────────────────────────────────────────────────┘
                          │
                          ↓
┌──────────────────────────────────────────────────────────┐
│  Application Layer (应用层)                               │
│  FastAPI Backend + Business Logic                         │
└──────────────────────────────────────────────────────────┘
                          │
                          ↓
┌──────────────────────────────────────────────────────────┐
│  Data Layer (数据层)                                      │
│  Supabase (PostgreSQL + Auth + RLS)                       │
└──────────────────────────────────────────────────────────┘
                          │
                          ↓
┌──────────────────────────────────────────────────────────┐
│  Storage Layer (存储层)                                   │
│  S3-compatible Object Storage                             │
└──────────────────────────────────────────────────────────┘
```

3.3 技术栈详解

前端技术栈

技术	版本	用途
Next.js	16	React全栈框架，App Router
React	19	UI库
TypeScript	Latest	类型安全
Tailwind CSS	Latest	样式框架
Radix UI	Latest	无障碍组件库
TanStack Query	Latest	数据获取和缓存
Zustand	Latest	状态管理

特性：

- 🚀 服务端渲染（SSR）
- ⚡ 自动代码分割

- 🎨 响应式设计
- ♿ 无障碍支持

后端技术栈

技术	版本	用途
FastAPI	Latest	Web框架
Python	3.11+	编程语言
asyncpg	Latest	异步PostgreSQL驱动
aioboto3	Latest	异步AWS SDK
Pydantic	Latest	数据验证
uvicorn	Latest	ASGI服务器

特性：

- 🔥 异步处理
- 📚 自动API文档（Swagger）
- ⚡ 高性能
- 🔒 类型安全

数据库与存储

组件	技术	用途
数据库	PostgreSQL	结构化数据
全文搜索	PGroonga	中英文搜索
对象存储	S3兼容存储	文件存储
认证	Supabase Auth	OAuth + JWT
实时	Supabase Realtime	WebSocket订阅

MCP服务器

核心组件：

组件	技术
SDK	MCP Python SDK
协议	Model Context Protocol
认证	Supabase OAuth
传输	HTTP/WebSocket

提供的5个工具：

工具	功能	参数
`guide`	解释Wiki工作原理，列出知识库	无
`search`	搜索文档（PGroonga排序）	query, kb_id
`read`	读取文档（支持页码范围）	doc_id, pages
`write`	创建/编辑Wiki页面	path, content, mode
`delete`	归档文档	paths

3.4 核心功能

功能列表

功能模块	功能	说明
用户管理	注册、登录、OAuth	Email + Google/GitHub
知识库	创建、管理多个知识库	隔离数据
文档上传	拖放上传、批量上传	TUS协议断点续传
文档处理	PDF提取、Office转换、OCR	Mistral API
文档查看	内置PDF/HTML查看器	页面导航、搜索、缩放
Wiki浏览	目录树、搜索、链接跳转	Markdown渲染
全文搜索	中英文混合搜索	PGroonga
Claude集成	MCP连接	5个工具
版本历史	页面变更追踪	时间戳记录
协作	共享知识库（计划中）	权限管理

文档处理流程

```
上传文件
    ↓
┌──────────────┐
│ 文件类型？   │
└──────┬───────┘
       ↓
    ┌──┴──────────────────┐
    │PDF    │Office │HTML  │
    ↓       ↓       ↓
┌─────┐  ┌─────┐  ┌─────┐
│提取 │  │转PDF│  │解析 │
│文本 │  │后提取│  │HTML │
└──┬──┘  └──┬──┘  └──┬──┘
   │        │        │
   └────────┴────────┘
            ↓
    ┌──────────────┐
    │ 有文本吗？   │
    └──────┬───────┘
      是 ↓   ↓ 否
         │   └──────→ OCR (Mistral API)
         │                  ↓
         └──────────────────┘
                   ↓
         ┌──────────────────┐
         │ 保存到数据库      │
         │ + S3存储         │
         └──────────────────┘
                   ↓
         ┌──────────────────┐
         │ 通知Claude可用    │
         └──────────────────┘
```

MCP工作流程

```
Claude.ai 用户提问
    ↓
Claude调用 search 工具
    ↓
MCP Server → Supabase (PGroonga搜索)
    ↓
返回匹配文档列表
    ↓
Claude调用 read 工具
    ↓
MCP Server → S3读取内容
    ↓
返回文档内容
    ↓
Claude分析并回答
    ↓
Claude调用 write 工具（可选）
    ↓
MCP Server → 保存Wiki页面到S3 + 更新数据库
    ↓
完成
```

3.5 部署选项

自托管（推荐团队/企业）

部署方式：

方式	复杂度	控制度	成本
Docker Compose	⭐⭐	⭐⭐⭐	低
Kubernetes	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	中
Serverless	⭐⭐⭐	⭐⭐⭐	低到中

3.6 模型付费

Claude使用

套餐	费用	特性
Claude Pro		消息限制较少，优先访问新模型
Claude API		更灵活，适合重度使用

估算成本：

3.7 自托管硬件要求

小规模部署（<50用户，<5000文档）

组件	配置
CPU	4核
内存	8GB
存储	100GB SSD
数据库	自托管PostgreSQL (2GB内存)
对象存储	MinIO (本地) 或 Cloudflare R2 (10GB免费)
网络	10Mbps上行

中规模部署（50-500用户，5000-50000文档）

组件	配置
Web/API服务器	8核，16GB内存，200GB SSD
数据库	PostgreSQL (8GB内存)
对象存储	500GB
负载均衡	可选
网络	100Mbps

估算成本：

大规模部署（>500用户，>50000文档）

组件	配置
Web服务器	2-4台，8核16GB
API服务器	2-4台，8核32GB
数据库	自建集群
对象存储	5TB+
CDN	Cloudflare/AWS CloudFront
负载均衡	必需
监控	Datadog/Prometheus

估算成本：

3.8 优缺点分析

优势

优势	说明
✅ **Web界面**	现代化UI，非技术用户友好
✅ **多用户**	支持团队协作
✅ **强大搜索**	PGroonga中英文全文搜索
✅ **内置查看器**	PDF/HTML在线查看
✅ **MCP原生**	与Claude深度集成
✅ **扩展性强**	可处理大规模数据
✅ **企业级**	完整认证、权限、日志
✅ **自动备份**	数据库备份（Supabase）
✅ **移动友好**	响应式设计

劣势

劣势	说明
❌ **复杂度高**	需要配置多个服务
❌ **成本较高**	服务器+数据库+存储
❌ **维护负担**	需要运维知识
❌ **学习曲线**	技术栈复杂
❌ **依赖网络**	完全依赖网络连接
❌ **不可离线**	无离线模式

第四部分：方案对比与选型建议

4.1 全面对比表

基础特性对比

维度	LLM-Wiki Skill	LLM-Wiki开源实现
类型	Agent技能/CLI工具	全栈Web应用
界面	无UI（Obsidian查看）	现代化Web UI
安装复杂度	⭐⭐ 简单	⭐⭐⭐⭐ 复杂
技术门槛	Mac/Linux	中等
部署方式	本地安装	自托管

数据存储对比

维度	LLM-Wiki Skill	LLM-Wiki开源实现
原始资料	本地文件系统	S3对象存储
Wiki页面	Markdown文件	S3 + 数据库元数据
数据库	无（文件系统）	PostgreSQL
备份	Git + 手动备份	自动数据库备份
版本控制	Git原生支持	数据库时间戳
数据隐私	完全本地	完全本地

功能对比

功能	LLM-Wiki Skill	LLM-Wiki开源实现
文档上传	命令行/手动	拖放Web上传
文档处理	外部工具	内置处理器
文档查看	Obsidian	内置PDF/HTML查看器
全文搜索	索引文件（小规模）	PGroonga（大规模）
Claude集成	通过Schema指令	原生MCP服务器
多用户	❌ 单用户	✅ 多用户
权限管理	❌ 无	✅ 完整RLS
移动端	Obsidian移动版	响应式Web
协作	Git（技术门槛高）	内置（计划中）

扩展性对比

维度	LLM-Wiki Skill	LLM-Wiki开源实现
最大文档数	~1000（索引限制）	数万（数据库支持）
并发用户	1	数百到数千
搜索性能	线性扫描	索引加速
存储上限	磁盘空间	S3（无限）
扩展方式	添加工具脚本	微服务架构

开发维护对比

维度	LLM-Wiki Skill	LLM-Wiki开源实现
初始搭建	1小时	
日常维护	几乎为0	
更新升级	Git pull	
备份策略	Git自动	需要配置
故障排查	简单（日志文件）	复杂（多服务协同）
定制难度	低（Bash脚本）	高（需要Web开发能力）

文档结束